

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 10
Flow Control

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Flow Control (Go-Back N) socket programming in C

server.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/in.h>
#include <ctype.h>
#include <netdb.h>
#define MAX_PENDING 5
#define MAX_LINE 256

int main(int argc, char **argv){
    struct sockaddr_in sin;
    int pos,len,s,new_s,ack=0;

    char transmit[4] = "no";
    char buffer[MAX_LINE],ch;
    //char sentback[50];
    bzero((char *)&sin, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_addr.s_addr = INADDR_ANY;
    sin.sin_port = htons(atoi(argv[1]));

    /* setup passive open */
    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {
```

```
perror("simplex-talk: socket");
```

```
exit(1); }
```

```
if ((bind(s, (struct sockaddr *)&sin, sizeof(sin))) < 0)  
{
```

```
perror("simplex-talk: bind");
```

```
exit(1); }
```

```
listen(s, MAX_PENDING);
```

```
if ((new_s = accept(s, (struct sockaddr *)&sin, &len)) <  
0) {
```

```
perror("simplex-talk: accept");
```

```
exit(1); }
```

```
printf("Established connection with client on port %d\n",  
atoi(argv[1]));
```

```
while (((len=recv(new_s,buffer,sizeof(buffer),0))>0)){  
    printf("Received: %s\n",buffer);  
    pos = buffer[4] - '0';  
    printf("pos:%d\n",pos);
```

```

        if(pos == ack){
            ack++;

        }
        printf("Sent ACK%d\n",ack);
        send(new_s,&ack,sizeof(int),0);

    }
    close(new_s);
    close(s);

}

```

//client.c

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/in.h>
#include <ctype.h>
#include <netdb.h>
#define MAX_PENDING 5

```

```

#define MAX_LINE 256

```

```

void packet_maker(char *str,char **packets,int rem,int q){
    char laststr[5]="";
    char packet_string[5]="";
    char zero = '0';

```

```

    int len,ind=0;
    len = strlen(str);
    //printf("%d\n",rem);
    int start = 0;
    for(int i =0;i<q;i++){

```

```

strcpy(packet_string, "");
for(int j=0;j<4;j++){

    strncat(packet_string,&str[start++],1);
}

strcpy(packets[ind++],packet_string);

//printf("\n");
}

if(rem!=0){
    int pos = 4 - rem;
    strcpy(laststr, "");
    for(int k=0;k<4;k++){

        if(k<pos){
            strncat(laststr,&zero,1);
        } else{

            strncat(laststr,&str[start++],1);
        }

    }

    strcpy(packets[ind++],laststr);
}

void retransmit(char **packets,int socketid,int from,int total)
{

    printf("Retransmitting!\n");
    for(int a=from;a<total;a++){

        send(socketid,packets[a],25,0);

    }

    printf("Sent packet %d\n",a);
}

```

```

    }

}

int main(int argc, char **argv) {

    char str[15],dst[2],src[2];
    char string[15]="";
    char msg[4]="yes";
    char transmit[4];

    char **packets;
    int total,length,rem,q,len,s,opt,ack=0,no;
    struct sockaddr_in sin;

    bzero((char *)&sin, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_port = htons(atoi(argv[1]));
    sin.sin_addr.s_addr = inet_addr(argv[2]);
    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {

        perror("simplex-talk: socket");

    exit(1); }

    if (connect(s, (struct sockaddr *)&sin, sizeof(sin)) < 0)
    {

        perror("simplex-talk: connect");
        close(s);
        exit(1);

    }
    printf("Established connection with server on port %d\n",atoi(argv[1]));

    strcpy(str,"");
    printf("Enter the message: ");
    scanf("%s",str);
    length = strlen(str);

```

```
printf("%d\n",length);
rem = length%4;
q=length/4;
if(rem>0){
```

```
total=q+1;
```

```
}else{
```

```
    total=q;
```

```
}
```

```
packets = (char **)malloc(total* sizeof(char *));
```

```
for (int i = 0; i < total; i++) {
```

```
    packets[i] = (char *)malloc(25 * sizeof(char));
```

```
}
```

```
packet_maker(str,packets,rem,q);
```

```
printf("Total Packets are %d\n",total);
```

```
printf("Enter the source: ");
```

```
scanf("%s",src);
```

```
printf("Enter the destination: ");
```

```
scanf("%s",dst);
```

```
for(int l=0;l<total;l++){
```

```
    strcpy(string,"");
```

```
    snprintf(string,sizeof(string),"F-%s-%d-%s-FCS-  
F",dst,l,packets[l]);
```

```
    strcpy(packets[l],string);  
}
```

```
for(int a=0;a<total;a++){  
    printf("Want to send packet %d[1=yes][0=no]: ",a);  
    scanf("%d",&opt);  
    if(opt==1){
```

```
        send(s,packets[a],25,0);  
        printf("Sent packet %d\n",a);  
        ack = packets[a][4] - '0';  
        recv(s,&no,sizeof(int),0);  
        printf("Received ACK%d\n",no);
```

```
}}
```

```
//printf("ack:%d no:%d\n",ack,no);  
if(ack+1!=no){
```

```
    retransmit(packets,s,no,total);
```

```
}  
  
}
```

O/P:

```
vboxuser@vm-vm: ~/Downloads  
vboxuser@vm-vm:~/Downloads$ gcc flow_serv.c -o flow_serv  
vboxuser@vm-vm:~/Downloads$ ./flow_serv 8080  
Established connection with client on port 8080  
Received: F-B-0-1100-FCS  
pos:0  
Sent ACK1  
Received: F-B-1-1100-FCS  
pos:1  
Sent ACK2  
Received: F-B-3-0000-FCS  
pos:3  
Sent ACK2  
Received: F-B-4-0000-FCS  
pos:4  
Sent ACK2  
Received: F-B-5-0011-FCS  
pos:5  
Sent ACK2  
Received: F-B-2-1110-FCS  
pos:2  
Sent ACK3  
vboxuser@vm-vm:~/Downloads$
```

```
vboxuser@vm-vm: ~/Downloads
vboxuser@vm-vm:~/Downloads$ gcc flow_cl.c -o flow_cl
flow_cl.c: In function 'main':
flow_cl.c:61:31: warning: implicit declaration of function 'inet_addr' [-Wimplicit-function-declaration]
   61 |         sin.sin_addr.s_addr = inet_addr(argv[2]);
      |                               ^
vboxuser@vm-vm:~/Downloads$ ./flow_cl 8080 10.0.2.15
Established connection with server on port 8080
Enter the message: 110011001110000000000011
24
Total Packets are 6
Enter the source: A
Enter the destination: B
Want to send packet 0[1-yes][0-no]: 1
Sent packet 0
Received ACK1
Want to send packet 1[1-yes][0-no]: 1
Sent packet 1
Received ACK2
Want to send packet 2[1-yes][0-no]: 0
Want to send packet 3[1-yes][0-no]: 1
Sent packet 3
Received ACK2
Want to send packet 4[1-yes][0-no]: 1
Sent packet 4
Received ACK2
Want to send packet 5[1-yes][0-no]: 1
Sent packet 5
Received ACK2
Retransmitting!
Sent packet 2
Sent packet 3
Sent packet 4
Sent packet 5
vboxuser@vm-vm:~/Downloads$
```

Learning Outcomes:

I learned how to implement flow control using socket programming.